

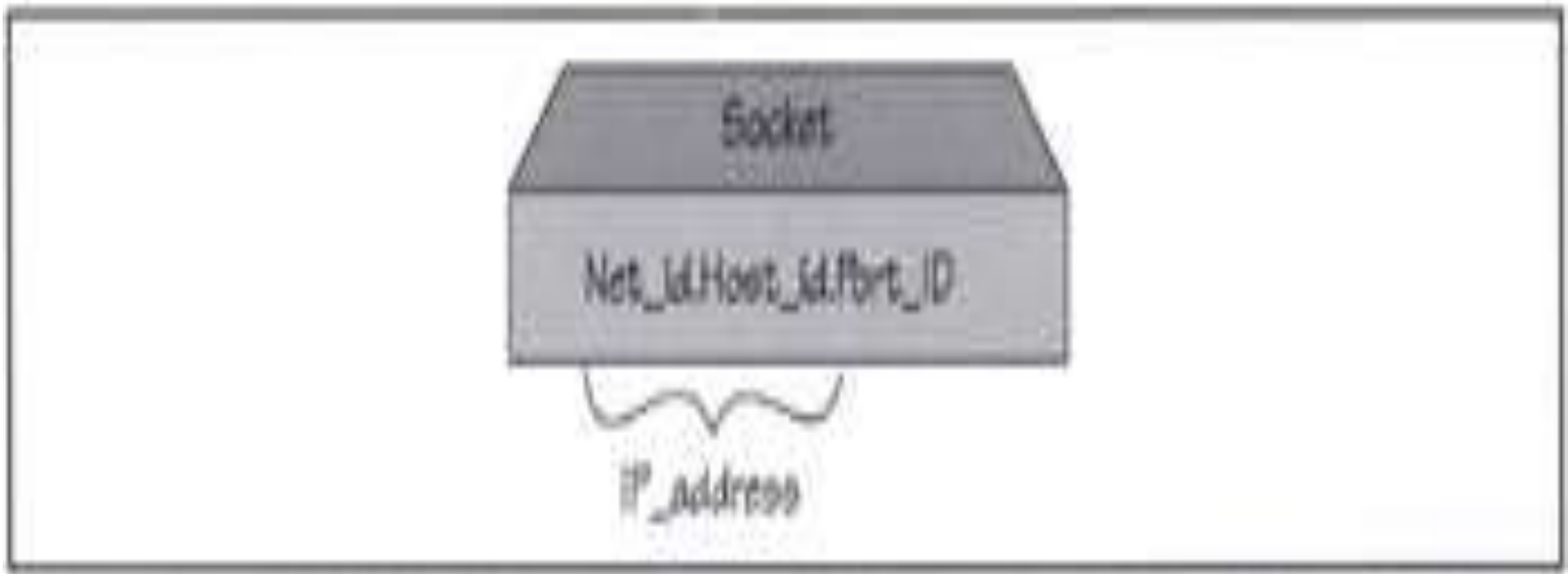
Multi Tier Computing

Types of Middleware

Peer-to-Peer Communications

- Most early client/server applications were implemented using low-level, conversational, peer-to-peer protocols—such as sockets, **TLI, CPIC/APPC, NetBIOS, and Named Pipes**
- programmers are now using RPCs, MOMs, and ORBs, which provide higher levels of abstraction
- So the conversational protocols should only be used by very demanding user applications or system-level software.
- The term "peer-to-peer" indicates that the two sides of a communication link use the same protocol interface to conduct a networked conversation.
 - **Any computer can initiate a conversation with any other computer.**
 - The protocol is **symmetrical**, and it is sometimes called "program-to-program."
 - **The peer-to-peer interface tends to be "close to the wire"** in the sense that it does not fully mask the underlying network from the programmer.
 - For example, the interface will surface transmission timeouts, race conditions, and network errors, and then leave it to the programmer to handle.

Sockets



- The three most popular **socket types** are **stream, datagram, and raw**.
- Stream and datagram sockets interface to the TCP and UDP protocols, and raw sockets interface to the IP protocol

NetBIOS and NetBEUI

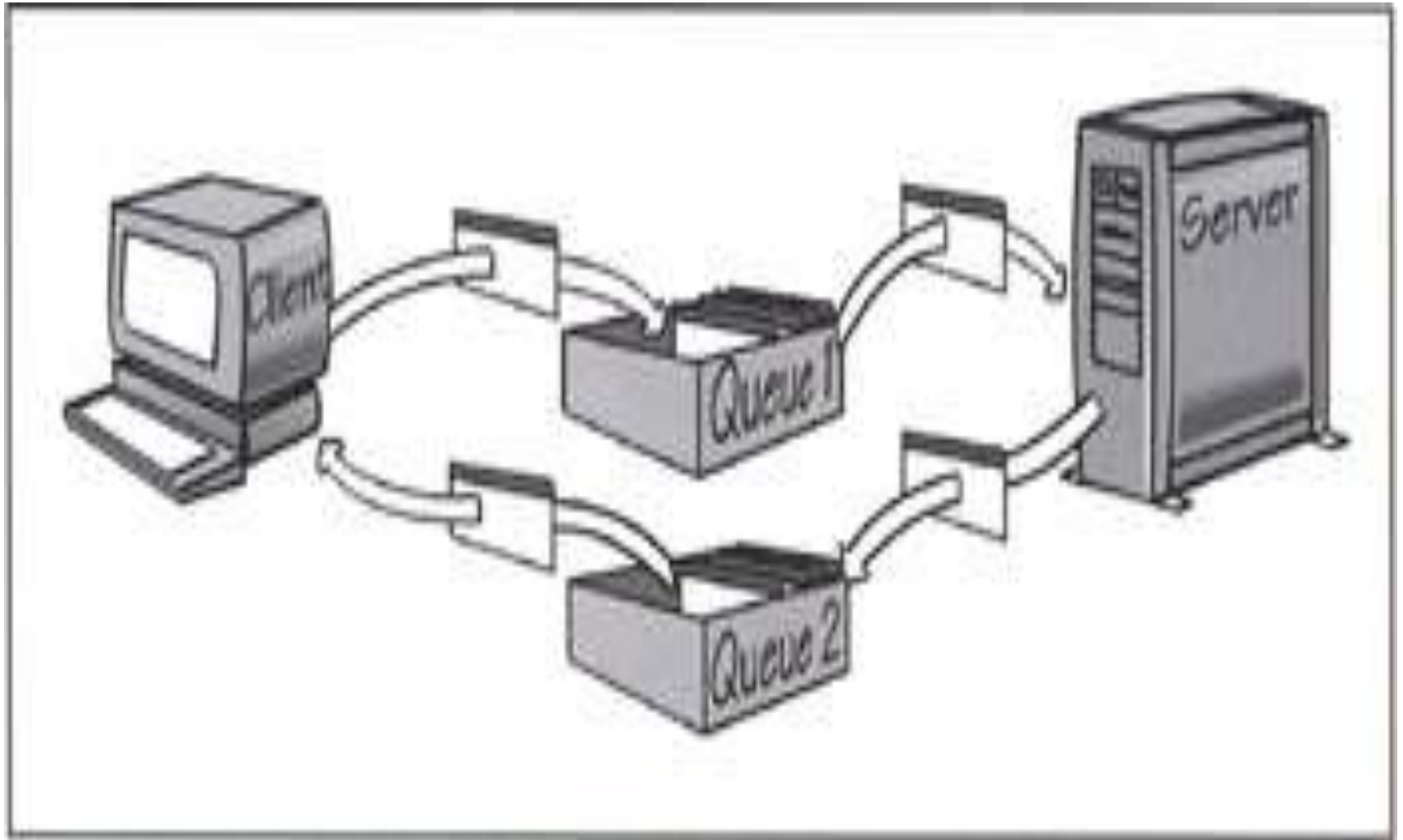
- NetBIOS is the premier protocol for **LAN-based, program-to-program communications**. Introduced by IBM and Sytek in 1984 for the IBM PC Network,
- **NetBIOS now runs with almost no changes on every LAN.**
- NetBIOS is **used as an interface to a variety of stacks**—including IBM/Microsoft LANs (NetBEUI), TCP/IP XNS, Vines, OSI, and IPX/SPX.
- Support for a NetBIOS platform exists on a multiplicity of operating system environments
- **NetBEUI is the protocol stack that comes with IBM and Microsoft LAN products**—including Windows for Workgroups, NT, LAN Manager, Windows 95/98, and OS/2 Warp

NetBIOS and NetBEUI

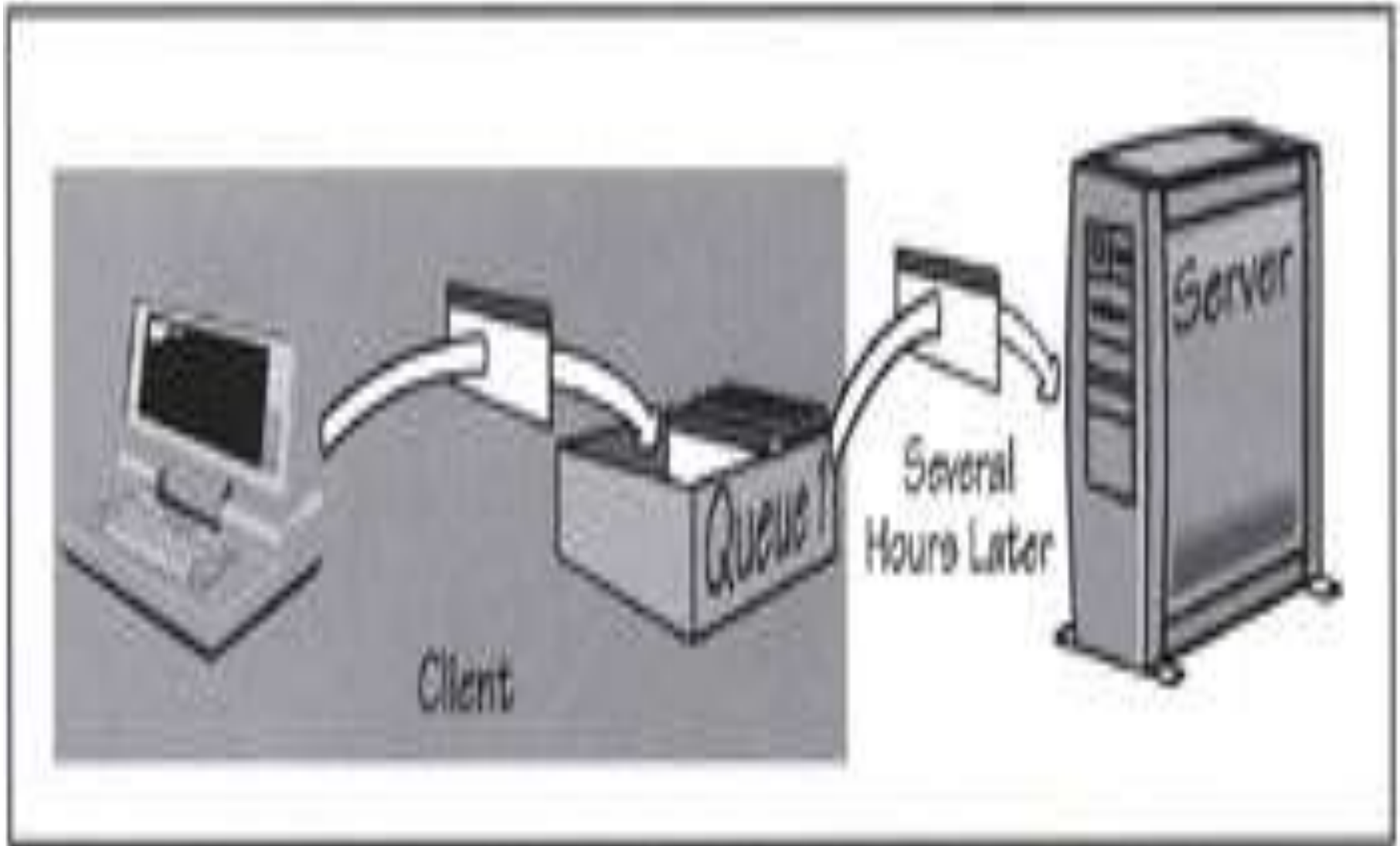
- **NetBEUI offers powerful datagram and connection-oriented services.**
- It also offers a **dynamic naming service** based on discovery protocols.
- NetBEUI's main weakness **is the lack of a network layer.**
- Its other weakness is **the lack of security.**
- The broadcast mechanism, used to dynamically "discover" names, can be a liability on an unsecured network where it's not a good idea to expose names.
- Broadcasting names also causes unwanted traffic on the network.
- Luckily, most bridges and routers have ways to filter the discovery packets and block them from propagating to other networks.

- MOM is a key piece of middleware that is absolutely essential for a class of client/server products
- If your application **can tolerate a certain level of time-independent responses**, MOM provides the easiest path for creating inter-enterprise client/server systems.
- MOM allows **general-purpose messages to be exchanged** in a client/server system using message queues.
- Applications communicate over networks by simply putting messages in queues and getting messages from queues
- MOM's messaging and queuing allow clients and servers to communicate across a network **without being linked by a private, dedicated, logical connection**
- MOM-enabled programs **do not talk to each other directly**, so either program can be busy, unavailable, or simply not running at the same time.
- A program **can decide when it wants to retrieve** a message off its queue
- Messaging queues are very versatile. You can use them to create one-to-many or many-to-one relationships

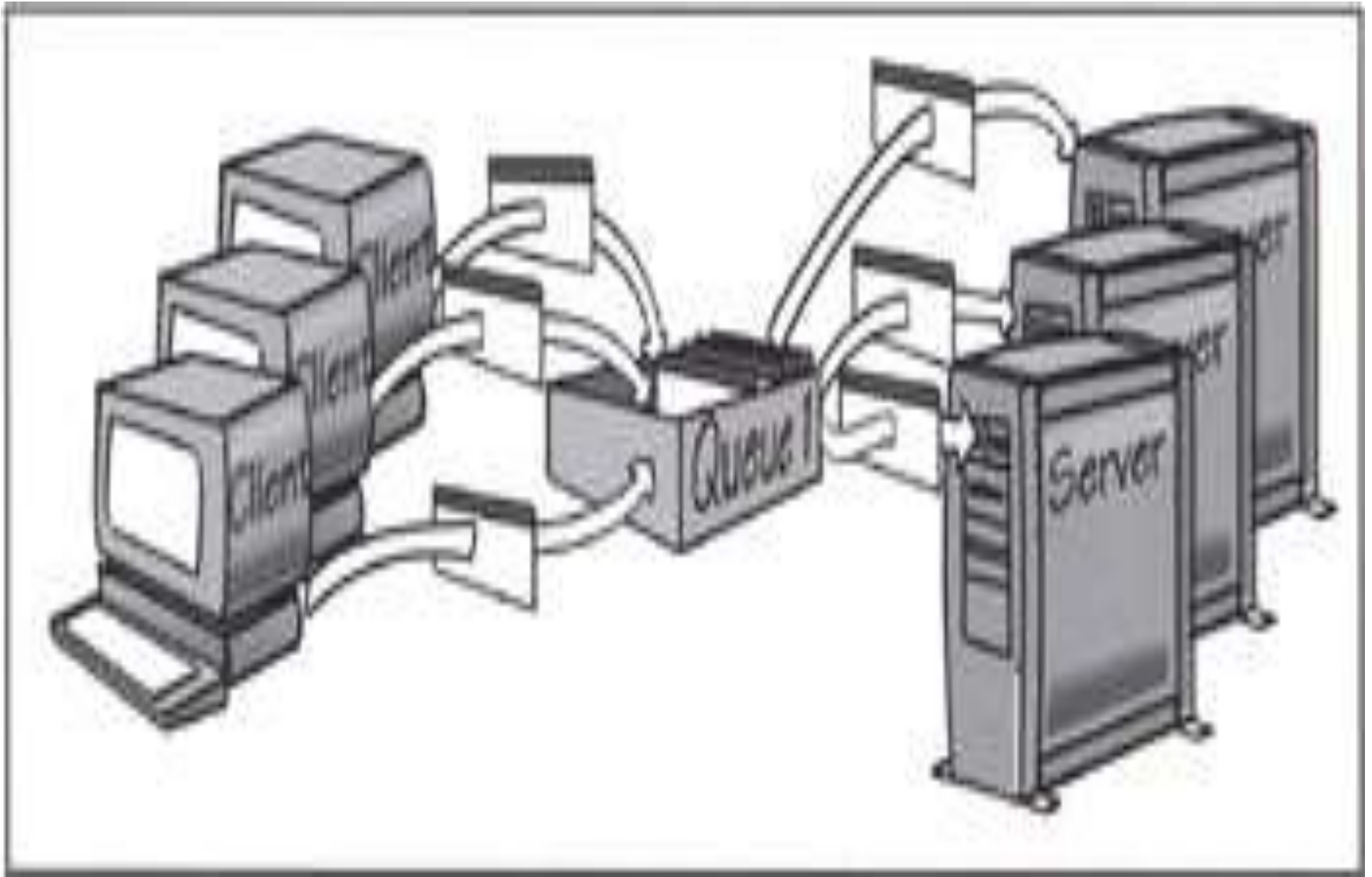
MOM: Two-way Message Queuing.



MOM: Save Your Messages Until You Get to a Server.



MOM: Many-to-Many Messaging via Queues.



- Most MOM messaging products make available a **simple API set that runs on multiple operating system** platforms
- Most also provide **persistent** (logged on disk) and **non-persistent** (in memory) message queues.
- Persistent messages are slower, but they can be recovered in case of power failures after a system restart. In both cases, messages can be either copied or removed from a queue.
- A **message queue** can be **local** to the machine or **remote**.
- System administrators can usually **specify the number of messages a queue can hold and the maximum message size**.

Mom Versus RPC

- Comparing the RPC and messaging paradigms is like doing business via a telephone call versus exchanging letters or faxes



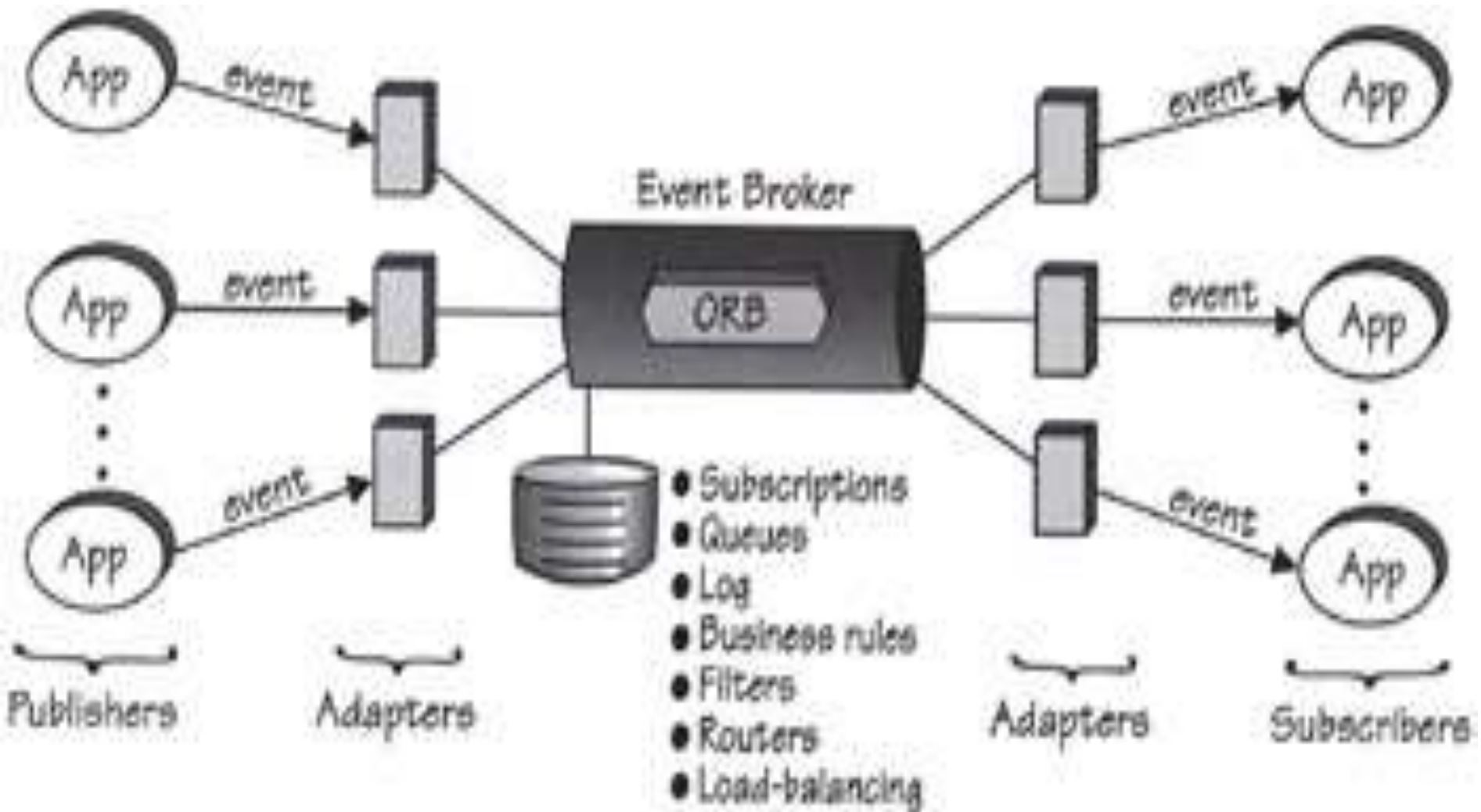
Remote Procedure Call



Messaging and Queuing

Feature	MOM: Messaging and Queuing	Remote Procedure Call (RPC)
Metaphor	Post office-like.	Telephone-like.
Client/Server time relationship	Asynchronous. Clients and servers may operate at different times and speeds.	Synchronous. Clients and servers must run concurrently. Servers must keep up with clients.
Client/Server sequencing	No fixed sequence.	Servers must first come up before clients can talk to them.
Style	Queued.	Call-Return.
Partner needs to be available	No.	Yes.
Load-balancing	Single queue can be used to implement FIFO or priority based policy.	Requires a separate TP Monitor.
Transactional support	Yes (some products). Message queue can participate in the commit synchronization.	No. Requires a transactional RPC.
Message filtering	Yes.	No.
Performance	Slow. An intermediate hop is required.	Fast.
Asynchronous processing	Yes. Queues and triggers are required.	Limited. Requires threads and tricky code for managing threads.

Publish and Subscribe?





THANK YOU